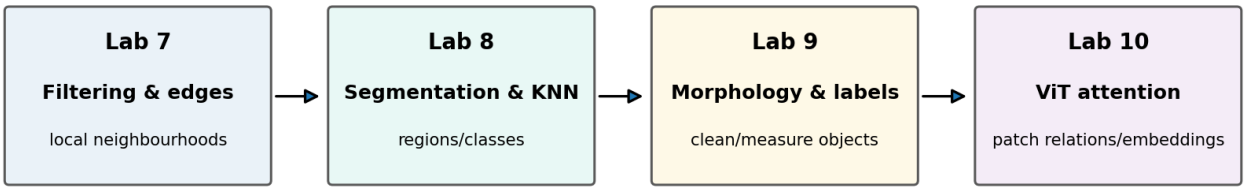
DIGITAL IMAGE PROCESSING (DIP)

Post-Mid-Sem Detailed Handout

Classification, Segmentation, Morphology, Connectivity, Transformers, ViT & GPT-style Mechanisms

Integrated with Labs 7-10 (MATLAB + Python/Colab) and course-generated visual material

Post-mid-sem learning progression



Scope and source note. The Lab 7-10 sections preserve the terminology and workflow of the supplied course materials. Sections on semantic/instance segmentation, transformer internals and GPT-style decoder attention are technical extensions added to make the handout complete; they are clearly connected back to the taught material.

Prepared for classroom revision, laboratory interpretation and project preparation

Post-Mid-Sem Concept Dashboard

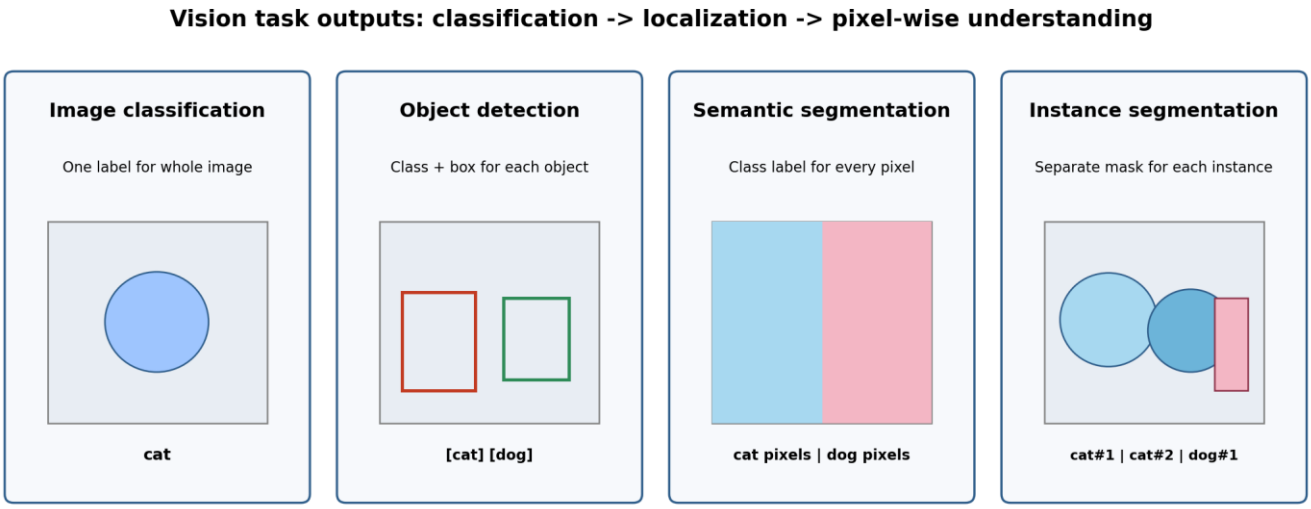


Course roadmap previously generated for the DIP sequence; the present handout extends the post-mid-sem portion.

Lab / Topic	Core idea	Typical input	Key output	Bridge to AI/CV
Lab 7 - Filtering & Edges	Neighbourhood filtering; gradients; Canny	gray/synthetic/noisy images	smoothed image, Gx/Gy, magnitude, edge map	CNNs learn local kernels/features
Lab 8 - Segmentation & Classification	K-means, region growing, pixel KNN	peppers.png, cameraman, optional selfie	cluster labels, binary regions, 4-class map	classical segmentation -> supervised ML
Lab 9 - Morphology & Components	shape cleanup + connectivity + region measures	synthetic binary masks, coins.png	clean mask, labels, area/centroid/bbox/perimeter	object-level representation before learned models
Lab 10 - ViT	patch tokens + self-attention + embeddings	224x224 photograph	attention matrices, patch features, retrieval	global/contextual visual representation

Big picture. Classical DIP makes image structure explicit (edges, regions, masks, geometry). CNNs learn local filters. Vision Transformers turn patches into tokens and learn relationships among them. These are complementary levels of representation, not mutually exclusive replacements.

1. From Image Classification to Pixel-Level Understanding



The word “segmentation” is often used loosely. For technically correct interpretation, distinguish the output required by the task:

Task	What is predicted?	Spatial output	Separates same-class instances?	Example
Image classification	One or more labels for the whole image	No explicit location	No	“This image contains a cat.”
Object detection	Class + bounding box for each object	Boxes	Yes	two cars -> two boxes
Semantic segmentation	Class label for every pixel	Dense class map	No	all road pixels share “road”
Instance segmentation	Class + separate pixel mask for each instance	Dense instance masks	Yes	person #1, person #2
Panoptic segmentation	Semantic map + instance identities for countable objects	Dense unified map	Yes for “things”	road + sky + separate cars

Terminology clarification. If “instant identification” is intended to mean identifying individual object instances, the standard terms are instance identification / instance segmentation. In the classical Lab 9 pipeline, connected-component labelling gives each disconnected foreground component an identity, but it does not by itself assign a semantic class.

1.1 Image classification

Image classification maps an input image x to a class label or class-probability vector. In a learned classifier, the model extracts features and then estimates p(class | image). The spatial extent of the object is not part of the required output.

- **Classical rule-based classification:** manually choose features such as area, perimeter, circularity, corners or texture, then apply explicit rules.
- **KNN classification:** store labelled feature vectors and classify a new vector from nearby training samples.
- **CNN classification:** learn convolution filters and hierarchical features jointly from labelled training data.
- **ViT classification:** use patch tokens and Transformer blocks; a classification head can use the [CLS] representation or pooled token features.

1.2 Segmentation

Segmentation partitions an image into meaningful pixel groups. Classical methods may group pixels by intensity, colour, connectivity or geometry; learned semantic/instance segmentation predicts pixel labels from data. A segmentation mask may be binary (foreground/background) or multi-class.

Critical distinction. A cluster is not automatically an object, and an edge map is not automatically a segmentation. K-means can assign the same cluster label to spatially disconnected regions; edge detection only highlights intensity transitions. Object identity requires additional spatial/semantic reasoning.

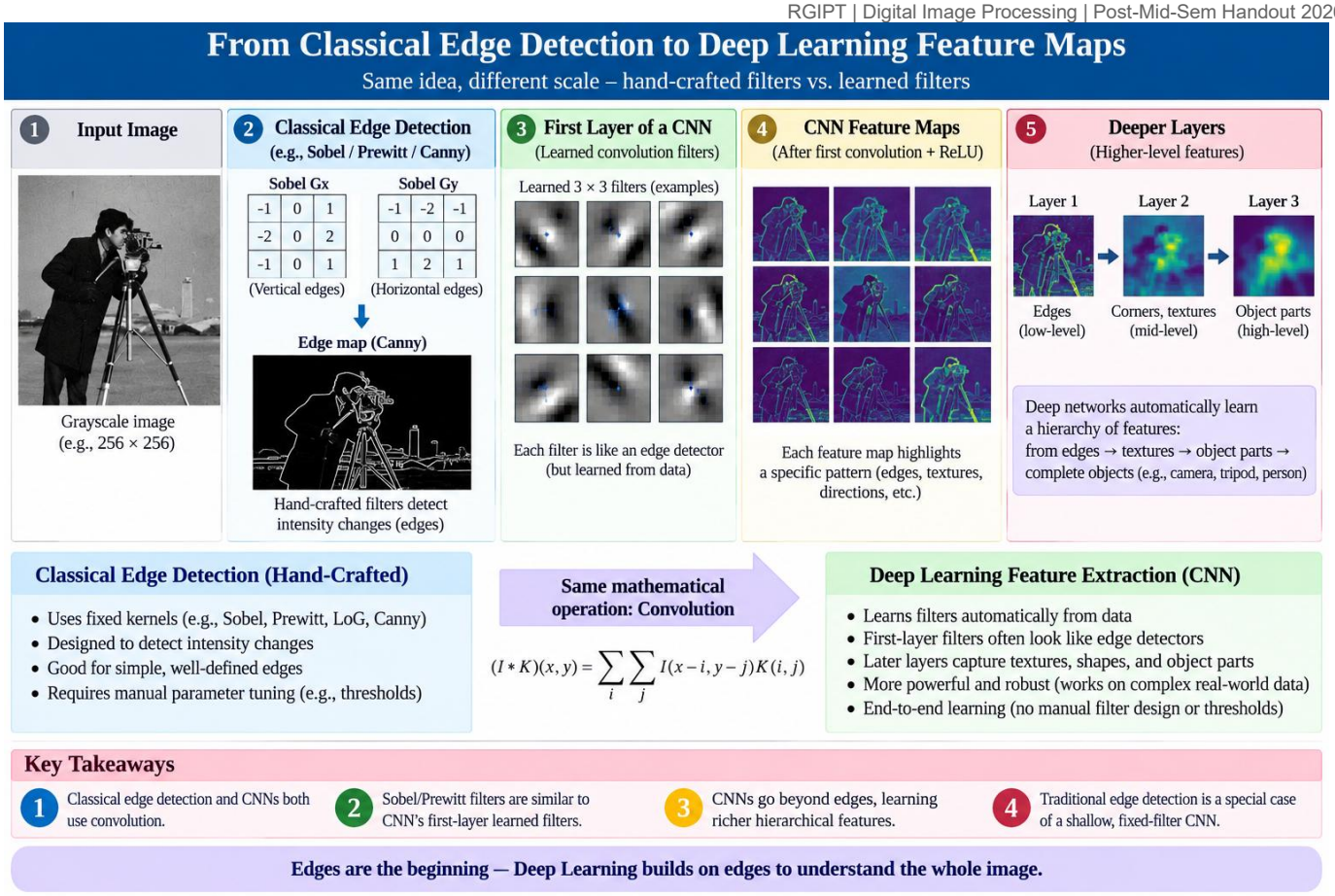
2. Lab 7 - Spatial Filtering, Gradients and Edge Evidence

Lab 7 provides the local-neighbourhood foundation for later segmentation. Smoothing reduces unwanted variation; derivative/high-pass operators emphasize transitions that may correspond to boundaries.

Method	Type	Core operation	Strength	Main limitation/use
Mean	linear low-pass	uniform average	simple noise smoothing	blurs edges/details
Gaussian	linear low-pass	weighted average	better local smoothing	still blurs; sigma/size matter
Median	nonlinear order-statistic	median of neighbourhood	excellent for impulse noise	not linear/convolutional
Mode	nonlinear	most frequent value	useful in discrete/categorical-like data	can be unstable on continuous gray values
Prewitt	1st derivative	Gx, Gy	simple gradient	more noise-sensitive than Sobel
Sobel	1st derivative + smoothing weight	Gx, Gy with central weight 2	robust classical gradient	directional derivative response
Laplacian	2nd derivative	nabla^2 I	direction-independent rapid-change response	amplifies noise
LoG	Gaussian then Laplacian	smooth + 2nd derivative	better noise control than raw Laplacian	scale dependent
Canny	multi-stage detector	smooth -> gradient -> NMS -> thresholds -> hysteresis	thin, connected edges	threshold/scale selection

```
% Sobel gradient and edge strength/direction
Sx = [-1 0 1; -2 0 2; -1 0 1];
Sy = [-1 -2 -1; 0 0 0; 1 2 1];
Gx = imfilter(I,Sx);
Gy = imfilter(I,Sy);
G = sqrt(Gx.^2 + Gy.^2);
Theta = rad2deg(atan2(Gy,Gx));
```

Technical clarification for MATLAB. The Lab 7 material describes neighbourhood filtering as convolution, but MATLAB imfilter(I,K) performs correlation by default. To request mathematical convolution explicitly, use imfilter(I,K,'conv'). For symmetric kernels (mean, Gaussian, many Laplacians) the result is the same; for asymmetric derivative kernels, sign/orientation can differ. Deep-learning libraries also commonly implement cross-correlation while calling the layer 'convolution'.



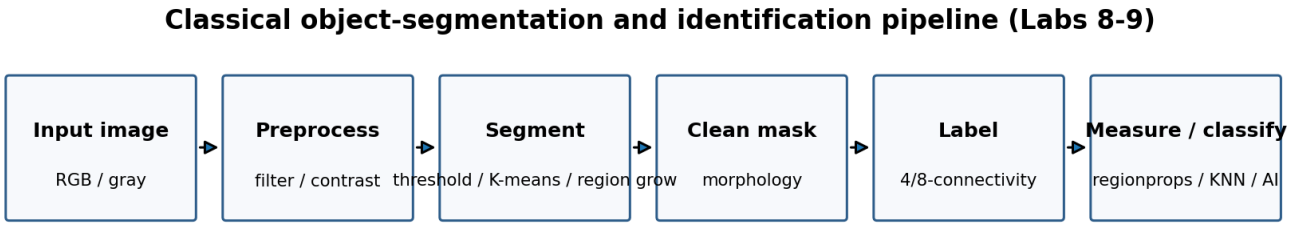
Previously generated course visual: fixed classical edge kernels versus learned CNN feature maps.

2.1 Canny edge detector: why it is more than "Sobel + threshold"

1. Gaussian smoothing reduces noise before differentiation.
2. Gradient components give magnitude and direction.
3. Non-maximum suppression keeps only local maxima along the gradient direction, producing thin candidates.
4. Double threshold separates strong, weak and rejected edge pixels.
5. Hysteresis keeps weak pixels that are connected to strong edges and rejects isolated weak responses.

Segmentation connection. Edges provide boundary evidence. A complete object mask usually needs region information, connectivity, contour closure, thresholding, morphology or a learned segmentation model.

3. Classical Image/Object Segmentation - Lab 8



3.1 Threshold-based segmentation

A binary threshold produces a mask BW(x,y). For a bright-object convention:

BW(x,y) = 1 if I(x,y) > T, otherwise 0.

Method	Threshold definition	Best situation	Weakness
Manual global	one user-chosen T for entire image	stable illumination; known intensity separation	sensitive to lighting / subjective T
Otsu	one data-driven global T	roughly bimodal foreground/background histogram	fails when classes strongly overlap or illumination varies
Adaptive/local	T(x,y) varies with neighbourhood	uneven illumination / shadows	window/sensitivity choices can create local noise

Otsu chooses a threshold that maximizes between-class variance (equivalently minimizes weighted within-class variance for the two classes). Adaptive thresholding estimates a local threshold surface rather than a single scalar.

3.2 K-means image segmentation

K-means is unsupervised clustering. Each pixel (or pixel feature vector) is assigned to the nearest cluster centroid. In Lab 8, MATLAB imsegkmeans groups image pixels into K colour/intensity clusters and label2rgb visualizes the label matrix.

Objective: minimize sum of squared distances from samples to their assigned centroids: J = sum_k sum_{x in C_k} ||x - mu_k||^2.

Do not over-interpret labels. K-means label 1/2/3... is an arbitrary cluster identity, not a semantic name. One cluster can appear in disconnected places; increasing K can split one real object into several colour clusters.

```
L = imsegkmeans(RGB,5);
M = label2rgb(L);
imshow(M);
% L is the numerical cluster-label matrix; M is only an RGB visualization.
```

3.3 Seed-based region growing: grayconnected

Region growing starts from a seed and accepts spatially connected pixels whose gray value is sufficiently similar to the seed value. The result is a logical binary mask.

```
[x,y] = ginput(1);
BW = grayconnected(Gray, round(y), round(x), 20);
imshow(BW);
```

- **Why round?** ginput returns continuous screen coordinates; array row/column indices must be integers.
- **Why y before x?** images use matrix indexing Gray(row,column), while ginput returns x=column and y=row.

- **Tolerance:** for uint8 data, 20 permits a wider intensity range than 5; the accepted pixels must still be connected to the seed region.
- **Multiple seeds:** combining BW1 | BW2 merges selected regions into one binary foreground mask. If separate object identities are required, keep masks separate or build a label image.

3.4 KNN pixel classification (Lab 8)

The Lab 8 KNN experiment is supervised classification at the pixel level. Training samples are manually clicked from red pepper, green pepper, yellow pepper and background. Each sample is represented by three RGB features.

```
n = 5;
[x,y] = ginput(n);
Red = zeros(n,3);
for i=1:n
    Red(i,:) = double(reshape(RGB(round(y(i)),round(x(i)),:),1,3));
end

trainData = [Red; Green; Yellow; Background];
trainLabel = [ones(n,1); 2*ones(n,1); 3*ones(n,1); 4*ones(n,1)];
Mdl = fitcknn(trainData,trainLabel,'NumNeighbors',3);
X = double(reshape(RGB,[],3));
label = predict(Mdl,X);
L = reshape(label,size(RGB,1),size(RGB,2));
```

Limitation of pixel-wise KNN. RGB-only KNN ignores neighbourhood and object shape. Pixels of different objects can share similar colours, and illumination can change RGB values. Spatial smoothing, morphology, richer features, colour-space changes or learned models can improve robustness.

4. Morphological Operators - Shape Processing of Binary Masks

Morphological Operators — Quick Handout

Binary image operations using a 3 × 3 structuring element

1. Structuring Element (SE)

3 × 3 structuring element (all ones):

$$B = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

For erosion, the output center pixel is 1 only if all covered pixels are 1. For dilation, the output center becomes 1 if any required foreground overlap occurs.

2. Sample Binary Image A

$$A = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

3. Erosion

Shrinks foreground objects. Any 0 under the SE causes the output center to become 0.

$$A \ominus B = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

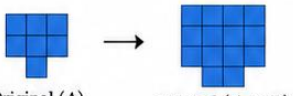


Original (A) → Eroded ($A \ominus B$) (shrinks)

4. Dilation

Grows foreground objects and can bridge small gaps.

$$A \oplus B = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$



Original (A) → Dilated ($A \oplus B$) (grows)

5. Opening

$$A \bullet B = (A \ominus B) \oplus B$$

Removes small protrusions/noise.

$$A \circ B = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$



Original (A) → Opened ($A \circ B$) (noise removed)

6. Closing

$$H \bullet B = (H \oplus B) \ominus B$$

Fills small holes and closes gaps.

H (with hole)

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$H \bullet B$ (closed)

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$



Original (H) → Closed ($H \bullet B$) (hole filled)

7. Fill (Hole Filling)

Changes enclosed 0s inside an object into 1s.

H (with hole)

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Filled result

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$



Original (H) → Filled (imfill) (hole filled)

8. Boundary Extraction

$$\beta(A) = A - (A \oplus B)$$

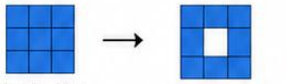
Original object minus its eroded version leaves the boundary.

A (5 × 5)

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Boundary $\beta(A)$

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$



Original (A) → Boundary ($\beta(A)$)

9. Summary Table

Operator	Formula	Main Effect
Erosion	$A \ominus B$	Shrinks object
Dilation	$A \oplus B$	Grows object
Opening	$(A \ominus B) \oplus B$	Removes small noise/protrusions
Closing	$(A \oplus B) \ominus B$	Fills small holes / closes gaps
Fill	imfill / hole filling	Fills enclosed holes
Boundary	$A - (A \oplus B)$	Extracts object boundary

10. MATLAB Commands

```
SE = strel('square',3);     % 3×3 structuring element (all ones)
E  = imerode(BW,SE);        % erosion
D  = imdilate(BW,SE);       % dilation
Op = imopen(BW,SE);         % opening
Cl = imclose(BW,SE);        % closing
F  = imfill(BW,'holes');    % hole filling
Bd = BW - imerode(BW,SE);   % boundary extraction
```

Course-generated morphology summary used in Lab 9; detailed definitions and caveats follow below.

Morphology operates on object geometry using a structuring element (SE). For a binary foreground set A and structuring element B, the SE is translated across the image. The output depends on both SE shape and size.

Operator	Set / operational idea	Main effect	Typical use
Erosion $A \ominus B$	keep center only when translated B fits inside A	shrinks foreground; enlarges holes/gaps	remove thin protrusions, separate narrow bridges
Dilation $A \oplus B$	foreground grows wherever translated/reflected B overlaps A	grows foreground; shrinks holes/gaps	bridge small gaps, connect near objects
Opening $A \circ B$	erosion followed by dilation	removes small foreground structures; smooths	remove small bright noise/protrusions
Closing $A \bullet B$	dilation followed by erosion	closes narrow breaks / small background gaps	join narrow cracks; fill small holes depending on SE
Hole fill	fill background components not connected to image border	fills enclosed holes	solid object masks
Boundary	$A - (A \ominus B)$	thin inner boundary	contour extraction
Morphological gradient	$(A \oplus B) - (A \ominus B)$	boundary band	shape/edge emphasis
Top-hat	$A - (A \circ B)$	small bright details removed by opening	uneven-background feature extraction
Black-hat	$(A \bullet B) - A$	small dark details closed by closing	dark defect / gap enhancement

4.1 Structuring element (SE)

- **Square SE:** isotropic for grid-aligned block-like neighbourhoods; larger size causes stronger erosion/dilation.
- **Disk SE:** approximately isotropic around the center; useful for rounded structures.
- **Line SE:** orientation-selective; useful for preserving/connecting line-like structures in a chosen direction.
- **MATLAB Neighborhood property:** the logical matrix showing which offsets belong to the SE.

```
SE3 = strel('square',3);
SEDisk = strel('disk',3,0);     % radius 3, exact/no decomposition
SELine = strel('line',7,0);     % length 7, angle 0 deg (horizontal)
imshow(SE3.Neighborhood);
```

4.2 Why erosion can be understood as a sliding logical test

For a flat 3x3 all-ones SE, move the SE center to each candidate foreground pixel. The output center is 1 only if every required SE position lies on foreground. If any required covered location is 0, that center becomes 0. This resembles sliding-window filtering geometrically, but it is not linear convolution: the operation is a set/min-type condition, not a weighted sum.

Grayscale morphology. For a flat structuring element, grayscale erosion behaves like a local minimum and grayscale dilation like a local maximum over the SE support. Binary morphology is the special case of set membership / 0-1 images.

5. Connectivity, Connected Components and Region Labelling - Lab 9

Neighbourhood	Included neighbours	Effect on diagonal contact
4-connectivity	up, down, left, right	diagonal-only pixels remain separate
8-connectivity	4-neighbours + four diagonals	diagonal-only pixels can belong to same component

If two foreground pixels touch only at a corner, they are separate under 4-connectivity but connected under 8-connectivity. Connectivity is a definition of neighbourhood; it does not enhance the image.

```
[L4,N4] = bwlabel(BW,4);
[L8,N8] = bwlabel(BW,8);
CC4 = bwconncomp(BW,4);
CC8 = bwconncomp(BW,8);
```

Labelling rule. The numeric labels produced by bwlabel identify connected components according to scan/labelling order, not object size. If you want the largest object to be label 1, sort components by area and relabel explicitly.

5.1 From mask to object measurements

```
stats = regionprops(L,'Area','Centroid','BoundingBox','Perimeter');
```

Property	Meaning	Use
Area	number of foreground pixels in region	size filtering; object comparison
Centroid	geometric center in image coordinates	tracking / annotation / location
BoundingBox	axis-aligned rectangle enclosing region	object localization / crop
Perimeter	estimated boundary length	shape descriptors / compactness

A common classical pipeline for coins/pills/bolts/cells is: grayscale -> threshold -> morphology -> remove small components -> connected components -> label -> regionprops -> count/measure/extract.

6. Semantic Segmentation, Instance Segmentation and Modern Object Identification

Technical extension beyond the supplied Labs 7-10. The labs build the classical foundations. The following section connects those foundations to modern dense prediction so that the terms semantic segmentation, instance segmentation and object identification are used correctly.

Method / family	Output	How object identity is represented	Typical architecture idea
Classical threshold / region grow	binary or label mask	connectivity / post-labelling	hand-designed rules
FCN / U-Net	semantic class per pixel	no same-class instance IDs	encoder-decoder / skip connections
DeepLab family	semantic class per pixel	no instance IDs	dilated/atrous context + decoder variants
Mask R-CNN	box + class + mask per detected instance	explicit instance per detection	detector + mask head
SAM-style promptable segmentation	mask(s) conditioned on prompt	mask proposals, not necessarily semantic class labels	image encoder + prompt encoder + mask decoder
Panoptic segmentation	semantic stuff + instance things	unified dense IDs	combined semantic + instance prediction

6.1 Losses and evaluation

For multi-class semantic segmentation, a common training loss is pixel-wise cross-entropy. For foreground/background masks, binary cross-entropy and Dice-style losses are common. Evaluation should not rely only on pixel accuracy when class imbalance is strong.

Metric	Formula / idea	Interpretation
IoU / Jaccard	TP / (TP + FP + FN)	overlap of predicted and true region
Dice / F1 mask	2TP / (2TP + FP + FN)	overlap with stronger weighting of intersection
Pixel accuracy	correct pixels / all pixels	can be misleading if background dominates
Mean IoU	average IoU across classes	standard semantic segmentation summary
Detection AP	precision-recall across IoU thresholds	object detection / instance segmentation evaluation

6.2 Classical-to-modern connection

- **Thresholding -> learned logits:** classical T is hand-chosen; a network learns a decision function from data.
- **Morphology -> post-processing:** opening/closing/connected components can still clean learned masks when application constraints justify it.
- **regionprops -> object analytics:** even after a neural mask, area, perimeter, centroid and shape statistics remain useful.
- **Edges -> boundary-aware learning:** gradient/boundary cues can be auxiliary losses/features in segmentation systems.

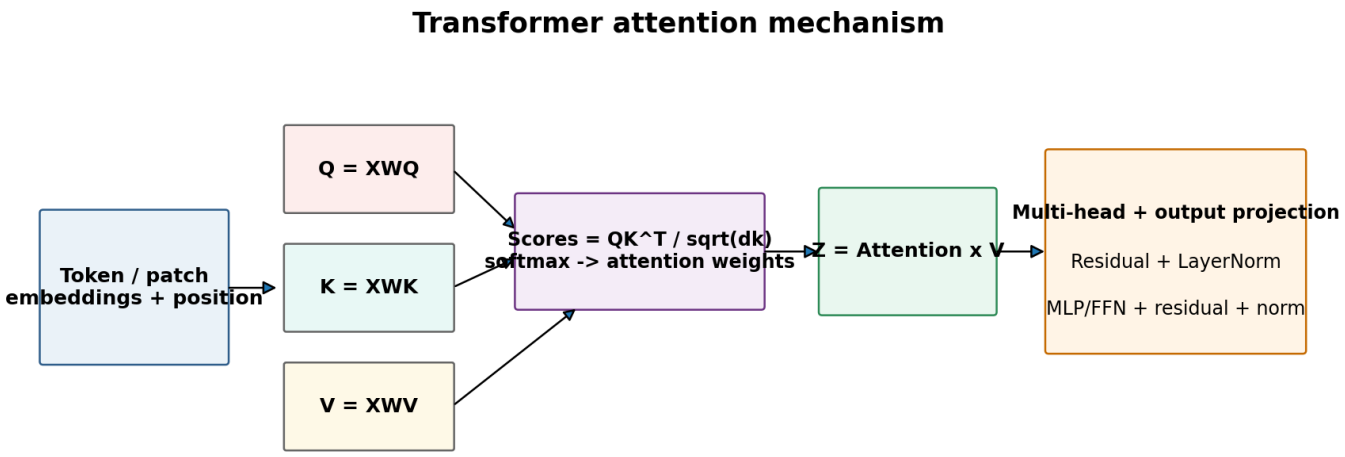
7. Image Classification: Classical Features, KNN, CNN and ViT

Approach	Features	Decision	Advantages	Limitations
Rule-based DIP	handcrafted geometry / colour / texture	explicit rules	transparent, exact geometry	fragile to variability
KNN	chosen feature vector (e.g., RGB)	nearest labelled samples	simple, no parametric training	slow at scale; sensitive to feature scaling
CNN	learned local convolutional hierarchy	trained classifier head	translation-aware local features; strong visual baseline	needs data/training; locality bias
ViT	learned patch-token representation + attention	classification head / pooled token	global relationships; scalable representations	attention cost grows with token count; needs suitable pretraining/data

CNNs replace fixed handcrafted kernels with learned weights. Early layers often respond to edges and orientations; deeper layers combine features into textures, parts and task-specific patterns. Feature maps are numeric activations, not automatically human-readable semantic labels.

Metric	Definition	When important
Accuracy	correct / total	balanced classes, overall performance
Precision	TP / (TP + FP)	false positives are costly
Recall	TP / (TP + FN)	missed positives are costly
F1	2PR / (P + R)	balance precision and recall
Confusion matrix	true class vs predicted class counts	see which classes are confused

8. Transformer Mechanism - What Attention Actually Computes



ViT encoder: typically bidirectional self-attention. GPT-style decoder: causal mask prevents attending to future tokens.

Let X be a sequence of token representations. A self-attention head computes learned linear projections:

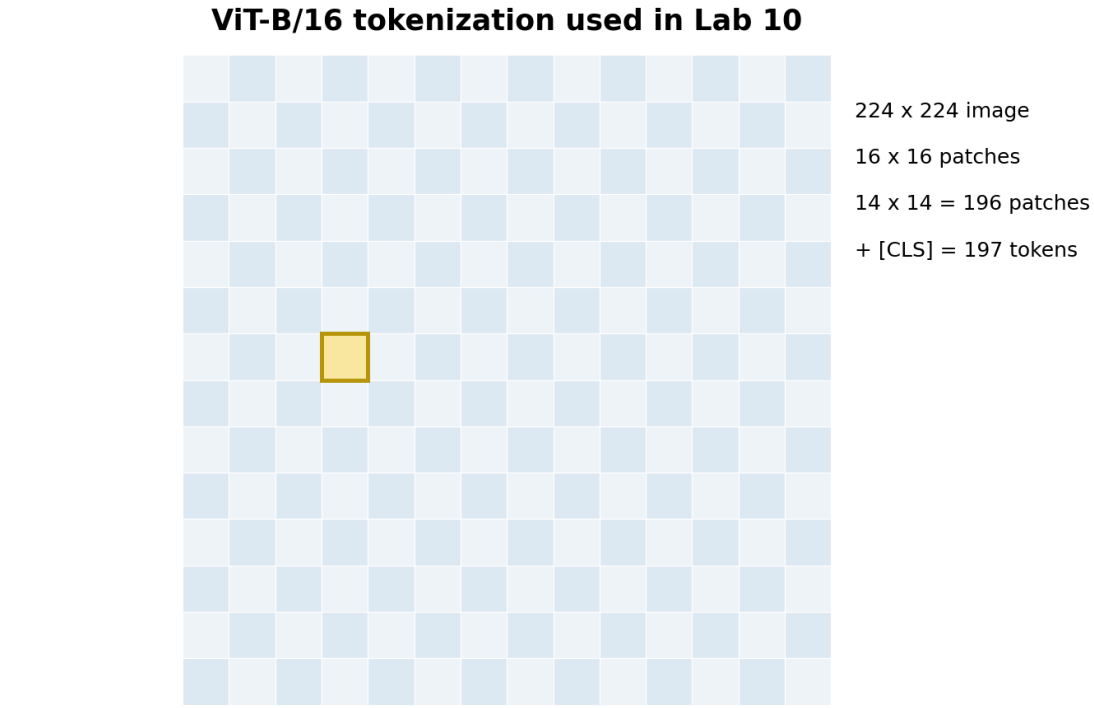
Q = XW_Q, K = XW_K, V = XW_V

Attention(Q,K,V) = softmax(QK^T / sqrt(d_k)) V

- **Query (Q):** what the current token is looking for in the context.
- **Key (K):** what each token offers for matching.
- **Value (V):** the information mixed after attention weights are computed.
- **Attention matrix:** row i contains the probability-like weights from query token i to all key tokens after softmax.
- **Multi-head attention:** several Q/K/V projections run in parallel, allowing different relation patterns; their outputs are concatenated/projected.
- **Transformer block:** attention sublayer + residual/norm + feed-forward/MLP sublayer + residual/norm (exact pre-norm/post-norm ordering depends on architecture).

Attention is not “meaning” by itself. Semantic knowledge is distributed across learned embeddings, attention/MLP weights and representations produced by many layers. A high attention value is an association used by that head/layer; it is not proof of causality, visual similarity or a named concept.

9. Vision Transformer (ViT) - Lab 10 in Detail



Each patch is flattened, linearly projected to an embedding, and combined with positional information.

The supplied Lab 10 notebook uses a pretrained ViT base model with 224x224 input and 16x16 patches. Therefore 14x14 = 196 image patches are formed. A special [CLS] token makes 197 tokens. The notebook analyses real returned attention tensors and contextual patch features.

Quantity	Lab 10 value / meaning
Input image	224 x 224 RGB
Patch size	16 x 16
Spatial patch grid	14 x 14
Image patch tokens	196
Special token	[CLS] = token 0 in the teaching notebook
Total tokens	197
ViT-base blocks	12 Transformer encoder blocks
Attention heads	12 per block
Attention matrix per head	197 x 197
Hidden embedding size	768 for ViT-Base
Per-head key/query dimension	768 / 12 = 64 (for standard ViT-Base configuration)

Patch vector vs embedding. A 16x16 RGB patch contains 16x16x3 = 768 raw numbers, but ViT still applies a learned patch-projection layer. The equality between raw patch dimension 768 and ViT-Base hidden size 768 is specific to this common configuration, not a general rule for all ViTs.

9.1 Positional information and [CLS]

Attention by itself is permutation-equivariant: it does not know that one patch is “top-left” and another is “bottom-right.” ViT adds learned positional embeddings (or another positional scheme) so spatial location is represented. The [CLS] token is a learned token intended to accumulate a global representation for tasks such as classification; it is not an image patch.

9.2 Layer and head interpretation

- **Head:** one parallel attention projection inside a block. Different heads can focus on different relationships.
- **Layer/block:** a sequential transformation stage. Layer 6 receives the representation produced by layers 1-5 and produces layer-6 features.
- **Selecting layer 6 in the notebook:** this selects the stored layer-6 attention for analysis; the normal forward pass still computes all 12 blocks unless the model is explicitly truncated or early-exited.
- **Selecting a query patch:** choose one row of the attention matrix; that row describes where that query token assigns attention across keys in that head/layer.

```
SELECT_LAYER = 12
SELECT_HEAD = 1
QUERY_TOKEN = 88
A = attentions[SELECT_LAYER-1][SELECT_HEAD-1]   # 197 x 197
query_row = A[QUERY_TOKEN].copy()
```

9.3 Attention vs feature similarity

The notebook deliberately separates two quantities:

Quantity	Computed from	Question it answers	Not equivalent to
Attention weight	softmax(QK^T/sqrt(dk)) inside one head/layer	“Which keys does this query attend to?”	pixel similarity / causal importance / cosine similarity
Cosine similarity	contextual hidden patch embeddings	“Which feature vector points in a similar direction?”	attention probability
RGB MSE / PSNR	pixel values	“How close are two patches numerically in RGB?”	semantic or embedding similarity

Why Lab 10 is retrieval, not generation. The notebook uses an intact query patch to retrieve a similar patch from other photographs, then substitutes it. It does not infer truly missing pixels from surrounding context. MAE/inpainting/diffusion-style models would be different because they learn a conditional reconstruction/generation objective.

10. GPT-Style Transformer vs ViT

GPT-style language models and ViT share the Transformer attention idea, but they use different tokenization and attention constraints. The key distinction is encoder-style bidirectional visual attention versus decoder-style causal language modeling.

Aspect	ViT encoder	GPT-style decoder-only Transformer
Token	image patch embedding (+ [CLS] in many designs)	text/subword token embedding
Position	2-D image position encoded into token sequence	1-D token order / positional representation
Self-attention	normally bidirectional: each token can attend to all tokens	causal: token t cannot attend to future tokens t+1...
Primary pretraining/task family	image representation/classification/self-supervised variants	next-token prediction / autoregressive generation
Typical output	image/patch representation; task head	probability distribution over next token
Mask	no causal mask in standard ViT encoder	triangular causal mask
Relation to prompts	not normally prompted with natural-language text by itself	prompt tokens condition subsequent generation

10.1 Where does the “meaning” of dog or ball come from?

A Transformer is the architecture. An LLM is a Transformer-based model whose parameters are learned from large-scale language training. Token embeddings start as learned vectors and become contextualized through repeated attention and MLP transformations. Meaning-like statistical/semantic structure is therefore distributed through the learned parameters and activations; there is not a separate “LLM” sitting inside each Transformer block.

10.2 What is the role of a prompt?

A prompt is the current input context and instruction. Its tokens are processed by the model and condition the activations and probability distribution for subsequent outputs. A prompt normally does not permanently retrain the model; it changes the current computation. In a text-plus-document question-answering setting, the inserted text supplies temporary context and the question constrains what answer should be generated.

More precise than “the prompt directs attention.” Attention patterns can change because the prompt changes the token sequence and representations, but the prompt is not a manual switch that explicitly tells an attention head where to look. The learned network determines the attention weights from Q and K at inference time.

10.3 VLM connection

A Vision-Language Model connects visual representations with language. Common designs encode an image into visual tokens/features and align/project them into a language model or use cross-attention/unified multimodal tokens. This allows image captioning, visual question answering and multimodal reasoning. A plain GPT-style text model does not directly “see” pixels unless a vision pathway is added.

11. Labs 7-10 - What to Remember, What to Vary, What to Measure

Lab	Core experiment	Change one thing at a time	Evidence/analysis
7	smoothing + edge operators	kernel type/size, Canny threshold, synthetic edge direction	noise removal vs blur; edge magnitude/direction; directional response
8	K-means, region growing, KNN	K clusters, seed/tolerance, KNN K and training points	cluster split/merge; region size; supervised vs unsupervised behaviour
9	SE morphology, thresholding, connectivity, labelling	SE shape/size, threshold method, area cutoff, 4 vs 8	foreground area; holes/gaps; component count; region measures
10	pretrained ViT attention + retrieval	layer, head, query token, repository	attention pattern; strongest associated patch; cosine retrieval; MSE/PSNR

11.1 Lab 7 compact MATLAB commands

```
Jmean = imfilter(double(I),ones(3)/9);
Jgauss = imfilter(double(I),[1 2 1;2 4 2;1 2 1]/16);
Jmedian = medfilt2(I,[3 3]);
BWcanny = edge(I,'canny',[0.05 0.20]);
```

11.2 Lab 8 compact MATLAB commands

```
L = imsegkmeans(RGB,K);
BW = grayconnected(Gray,row,col,tol);
Filled = imfill(BW,'holes');
Mdl = fitcknn(trainData,trainLabel,'NumNeighbors',Knn);
label = predict(Mdl,double(reshape(RGB,[],3)));
```

11.3 Lab 9 compact MATLAB commands

```
SE = strel('disk',2);
E = imerode(BW,SE); D = imdilate(BW,SE);
Op = imopen(BW,SE); Cl = imclose(BW,SE);
F = imfill(BW,'holes'); Clean = bwareaopen(F,50);
[L,N] = bwlabel(Clean,8);
stats = regionprops(L,'Area','Centroid','BoundingBox','Perimeter');
```

11.4 Lab 10 compact Python notebook logic

```
# 224x224 image -> 196 patch tokens + CLS
model = ViTModel.from_pretrained(MODEL_ID, attn_implementation='eager')
out = model(pixel_values=pixel_values, output_attentions=True, return_dict=True)
attentions = [a[0].detach().cpu().numpy() for a in out.attentions]
features = out.last_hidden_state[0].detach().cpu().numpy()
A = attentions[layer-1][head-1]     # [197,197]
query_row = A[query_token]
# repository retrieval uses cosine similarity between contextual patch features
```

12. End-to-End Examples: How the Concepts Work Together

Application	Possible pipeline	Why each stage matters
Count coins / pills / cells	gray -> Otsu/adaptive -> opening/closing -> fill -> bwareaopen -> 8-CC -> regionprops	threshold finds foreground; morphology cleans; connectivity creates object identities; regionprops measures/counts
Pepper colour map	RGB -> K-means or KNN -> optional morphology -> connected regions	colour separates classes; supervised KNN adds semantic labels; morphology removes isolated speckles
Selfie foreground	contrast/gray or colour features -> segmentation -> morphology -> component selection	illumination strongly affects threshold; mask cleanup and largest/target component selection can help
Road scene	enhancement -> edges/lanes + CNN/ViT -> semantic/instance segmentation	classical geometry and learned semantics can be fused
Remote-sensing change	registration/enhancement -> difference/features -> threshold/segmentation -> morphology -> connected regions -> AI verification	DIP creates measurable change candidates; AI adds robustness/semantics
Visual retrieval	resize/normalize -> ViT patch/image embeddings -> cosine similarity -> ranked retrieval	global/contextual learned representation rather than hand-designed similarity

12.1 A “dashboard” mindset for a DIP project

A dashboard is useful only if it exposes meaningful visual and quantitative evidence. A strong DIP dashboard should not hide the processing behind one final label. It should show input, intermediate stages, parameters, output and metrics.

Dashboard panel	Recommended content
Input	original image/video/frame, dimensions, channels, bit depth
DIP stage	enhanced/filtered image, histogram, edges or mask
Object/region stage	label image, bounding boxes, centroids, area/perimeter/count
AI stage	class probabilities, segmentation overlay, attention/embedding view where relevant
Evaluation	accuracy/F1/IoU/Dice/PSNR/SSIM/runtime as appropriate
Failure analysis	one success + one failure case with parameter/model explanation

13. High-Value Technical Clarifications / Common Mistakes

Statement	Correct interpretation
“K-means gives K objects.”	No. It gives K clusters; each cluster can be spatially disconnected.
“Multiple grayconnected seeds give separate labels.”	Not if masks are OR-combined; the result is binary unless separate masks/labels are retained.
“Opening fills holes.”	Not generally. Opening removes small foreground structures; closing can close small background gaps/holes; imfill fills enclosed holes.
“Erosion is convolution with a 3x3 ones mask.”	No. Erosion is a nonlinear/set-min operation; sliding-window geometry is similar but arithmetic is different.
“8-connectivity enhances diagonal edges.”	No. It changes the definition of connectedness.
“Edge = object boundary.”	An edge is local intensity change; object boundaries require grouping/segmentation.
“Semantic segmentation identifies each car separately.”	No. Same-class pixels share one class label; instance segmentation separates individual cars.
“Attention is similarity.”	Not necessarily. Attention is task/context-dependent Q-K weighting; cosine similarity of hidden features is a different calculation.
“Selecting ViT layer 6 makes the model run only six layers.”	Not in the standard full forward pass used in Lab 10; layer selection is for analysis unless the model is explicitly truncated.
“GPT is inside a Transformer.”	GPT is a model built from Transformer decoder blocks; the architecture is not a separate component inside an LLM.
“A prompt permanently teaches the model.”	Normally no; the prompt conditions the current inference context unless a separate training/memory mechanism is used.
“Lab 10 reconstructs a missing patch.”	No. It retrieves/substitutes a patch using features; true missing-patch prediction needs a generative/reconstruction model.

14. Quick Formula & Function Sheet

Topic	Key expression / function
Gradient magnitude	G = sqrt(Gx^2 + Gy^2)
Gradient direction	theta = atan2(Gy,Gx)
Canny	Gaussian -> gradient -> NMS -> double threshold -> hysteresis
Binary threshold	BW = (I > T)
K-means objective	min sum_k sum_{x in Ck} ||x - mu_k||^2
Erosion	A ⊖ B: translated B must fit inside A
Dilation	A ⊕ B: translated/reflected B overlaps A -> foreground growth
Opening	A ○ B = (A ⊖ B) ⊕ B
Closing	A • B = (A ⊕ B) ⊖ B
Boundary	beta(A) = A - (A ⊖ B)
IoU	TP / (TP + FP + FN)
Dice	2TP / (2TP + FP + FN)
Attention	softmax(QK^T / sqrt(d_k)) V
ViT-B/16 Lab 10	224/16 = 14 -> 196 patches + CLS = 197 tokens
Cosine similarity	u·v / (||u|| ||v||)

MATLAB / Python	Purpose
imfilter	spatial correlation by default; use conv option for convolution
edge(...,'canny')	Canny edge detection
imsegkmeans	K-means image clustering
grayconnected	seeded grayscale connected region
imfill(...,'holes')	fill enclosed holes
imerode / imdilate / imopen / imclose	binary/grayscale morphology
bwareaopen	remove small connected foreground components
bwlabel / bwconncomp	connected-component labelling/structure
regionprops	region measurements
fitcknn / predict	supervised KNN training/inference
ViTModel.from_pretrained	load pretrained ViT encoder
output_attentions=True	request attention tensors
last_hidden_state	contextual token embeddings/features

15. Revision / Viva Questions

6. Why can a 5x5 mean filter remove more noise but blur more detail than a 3x3 mean filter?
7. Explain the difference between gradient direction and edge orientation in Sobel-based analysis.
8. Why is non-maximum suppression needed in Canny?
9. Why can K-means produce visually coherent colour clusters but still fail to identify objects?
10. How does region growing differ from ordinary thresholding?
11. Why do multiple seeds combined with logical OR lose individual seed identities?
12. Give a case where erosion improves a mask and a case where it damages a valid object.
13. Why can closing connect two objects that should remain separate?
14. Construct a small binary example for which 4-connectivity gives two components but 8-connectivity gives one.
15. Distinguish semantic segmentation, object detection and instance segmentation using the same street image.
16. Why can high pixel accuracy coexist with poor segmentation IoU?
17. Why are learned CNN kernels related to but not identical in purpose to Sobel/Prewitt kernels?
18. Why are positional embeddings necessary in ViT?
19. Why does a 224x224 image with 16x16 patches produce 196 image tokens?
20. What does one row of a 197x197 attention matrix mean?
21. What is the difference between changing an attention head and changing a Transformer layer?
22. Why is ViT attention normally bidirectional while GPT-style attention is causal?
23. Why is attention weight not the same as cosine similarity?
24. What does a prompt change during GPT inference, and what does it not normally change permanently?
25. Why is Lab 10 patch substitution not true reconstruction of missing pixels?

16. Source Basis and Technical-Extension Note

This handout was synthesized from the supplied/previously generated course material and then technically expanded where requested. Source-derived sections retain the course terminology and laboratory sequence; extensions are labelled in the relevant sections.

- DIP Lab 7 - Image Filtering & Edge Detection using MATLAB.
- DIP Lab 8 - Classical Image Segmentation & Supervised Classification using MATLAB.
- Lab 9 - Morphological and Connectivity based Object Identification / student instruction.
- DIP Lab 10 - Real Vision Transformer Attention and Patch-Based Reconstruction (Python/Colab notebook).
- DIP Mid-Sem Compact Handout 2026 and course coverage roadmap.
- Course-generated Morphological Operators quick handout and Classical Edges -> CNN Features visual.
- Course project guideline material for end-to-end DIP -> AI/CV -> dashboard/evaluation connections.

Technical extensions added for completeness: formal distinction among classification/detection/semantic/instance/panoptic segmentation; IoU/Dice/AP overview; U-Net/DeepLab/Mask R-CNN/SAM-style positioning; explicit Transformer Q/K/V and causal-mask explanation; ViT vs GPT-style comparison; and precision notes about MATLAB correlation vs mathematical convolution.

Study strategy. First master the visible pixel operations (Labs 7-9), then connect them to learned representations (CNN/ViT). In every method, ask: What is the input? What transformation is applied? What parameter is adjustable? What is the output representation? How is performance measured?